

MirrorUI's Documentation

Introduction

This module allows the programmer to easily mirror touchpanels, network button panels, and eBUS panels in a programmed system.

The module consists of 7 classes: MirrorUIDevice, MirroreBUSDevice, Button, Knob, Label, Level and Slider. The MirrorUIDevice and MirroreBUSDevice classes are used to define or group the UIDevice objects and eBUSDevice objects as a single mirrored object. The mirrored object is then used to define the Button, Knob, Label, Level and Slider objects in the user interface. Once defined, the objects are programmed using the same properties, methods and events defined in the ExtronLib UIDevice, eBUSDevice, Button, Knob, Label, Level and Slider classes.

Version History

Version	Date	Description
2.0.0	3/11/2024	Major change to support new ControlScript Framework. Compatible with <i>eventEx</i> . Return values will be a single item for all things that are expected to be uniform across all objects in a mirror group. Others will continue to return a list. Removed MirroredParent.
1.2.1	4/22/2020	Fixed HDCPStatusChanged event.
1.2.0	9/24/2019	Added Slider object to MirrorUI module. Corrected use of Timer over Wait for periodic events.
1.1.1	1/28/2019	Fixed a missing duration parameter for MirrorUIDevice.ShowPopup.
1.1.0	12/27/2018	Added MirroredParent property to identify device and ui objects within events. Added ModuleVersion method to return the version of the MirrorUI Module events.
1.0.1	12/13/2018	Fixed a documentation typo.
1.0.0	12/06/2018	Initial version.

Table of Contents

[Minimum Software and Firmware Requirements](#)

[MirrorUI Examples](#)

[Appendix](#)

- [MirrorUI API Definition](#)

UI Host Classes:

- [MirrorUIDevice Class](#)
- [MirroreBUSDevice Class](#)

UI Object Classes:

- [Button Class](#)
- [Knob Class](#)
- [Label Class](#)
- [Level Class](#)
- [Slider Class](#)

Minimum Software and Firmware Requirements

IP Link Pro xi Series

Product	Firmware	ControlScript Pro xi
IPCP Pro xi Firmware	1.08.0000-b005	1.6.10
IPCP Pro Q xi Firmware	1.08.0000-b005	1.6.10
TouchLink Pro 1025/725/525 with LinkLicense	3.10.0000-b002	1.5.4
TouchLink Pro 300M with LinkLicense	1.05.0000-b002	1.5.4

IP Link Pro Series

Product	Firmware	ControlScript Pro
IPL Pro Firmware	3.17.0002-b002	3.13.18

Software

Software Product	Version
Global Scriptor Version	2.21.0
ControlScript Deployment Utility	1.4.0

MirrorUI Examples

Initial setup of mirrored group.

```
# Remove or comment out the extronlib.ui import statement
#from extronlib.ui import Button, Knob, Label, Level, Slider

# Add the import statement for the MirrorUI file in System Manager
from MirrorUI import MirrorUIDevice, Button, Knob, Label, Level, Slider

TLP1 = UIDevice( "PanelAlias" )
TLP2 = UIDevice( "PanelAlias_2" )

# Add the statement defining the MirrorUIDevice and program using normal
# methods and events
MirroredTLPs = MirrorUIDevice([TLP1, TLP2])
```

Use the mirrored group object as the host for UI Objects.

```
# Create an instance of button ID 101 for all panels
powerOn = Button(MirroredTLPs, 101)

# Creates an instance of label ID 3001 for all panels
RoomLabel = Label(MirroredTLPs, 3001)
```

Use the mirrored group UI Objects throughout the project.

```
@eventEx(powerOn, ['Pressed', 'Released'])
def PowerOnPressed(button, state):
    if state == 'Pressed':
        button.SetState(1)
    elif state == 'Release':
        TurnSystemOn()
        button.SetState(0)

@eventEx(MirroredTLPs, 'Online')
def MirroredTLPsOnline(device, state):
    RoomLabel.SetText('Lecture Hall')
```

A method for unique Setup Pages.

```
lblModel = Label(MirroredTLPs, 1001)
lblFirmwareVersion = Label(MirroredTLPs, 1002)
lblIPAddress = Label(MirroredTLPs, 1003)
lblMACAddress = Label(MirroredTLPs, 1004)

def Initialize():
    # to initialize the setup page on each panel
    # The for loop iterates for each panel in the MirroredTLPs object
    for Index, Panel in enumerate(MirroredTLPs.Objects):
        # The Index sets each object pulling data from the MirroredTLPs
        # property lists
        lblModel.Objects[Index].SetText(Panel.ModelName)
        lblFirmwareVersion.Objects[Index].SetText(Panel.FirmwareVersion)
        lblIPAddress.Objects[Index].SetText(Panel.IPAddress)
        lblMACAddress.Objects[Index].SetText(Panel.MACAddress)
```

Appendix

MirrorUI API Definition

Mirror UI module

Copyright 2018-2024, Extron. All rights reserved.

ModuleVersion Method

ModuleVersion()

This method returns the version of the Mirror UI Module

MirrorUIDevice Class

class **MirrorUIDevice**(*UIList*)

This class creates an object that contains a list of UIDevice objects to be used with the Button, Label, Level and Knob classes defined in this file. The class object is also used in place of the UIDevice object for touchpanel operations such as page flips and show/hide popup pages.

Initializes the module's MirrorUIDevice in program.

Parameters **UIList** (*list of extronlib.device.UIDevice*) – List of UIDevice Objects

Examples

```
# Comment out the extronlib.ui import statement
#from extronlib.ui import Button, Knob, Label, Level, Slider

# Add the import statement for the MirrorUI file in System Manager
from MirrorUI import MirrorUIDevice, Button, Knob, Label, Level, Slider

TLP1 = UIDevice( "PanelAlias" )
TLP2 = UIDevice( "PanelAlias_2" )

# Add the statement defining the MirrorUIDevice and program using normal
# methods and events
MirroredTLPs = MirrorUIDevice([TLP1, TLP2])
```

property **AmbientLightValue**

Defined Property to pass list of Objects' AmbientLightValue property

Returns list of current ambient light values for all devices
Return type list of int

property **AutoBrightness**

Defined Property to pass AutoBrightness

Returns current auto brightness state for all devices
Return type bool

property **Brightness**

Defined Property to pass Brightness

Returns current brightness level for all devices
Return type int

property **BrightnessChanged**

Passback the BrightnessChanged Event

Event: Triggers when LCD brightness has changed.

The callback takes two arguments. The first one is the MirrorUIDevice instance triggering the event and the second one is the current brightness level as an integer.

Examples

```
@event(MirroredTLPs, 'BrightnessChanged')
def HandleBrightnessChanged(mirroredtlp, brightness):
    print('{} Brightness Changed: {}'.format(mirroredtlp.DeviceAlias, brightness))
```

CallBrightnessChanged(*panel, brightness*)

Callback passed to device list to redirect event to new event call

CallHDCPStatusChanged(*panel, state*)

Callback passed to device list to redirect event to new event call

CallInactivityChanged(*panel, time*)

Callback passed to device list to redirect event to new event call

CallInputPresenceChanged(*panel, state*)

Callback passed to device list to redirect event to new event call

CallLidChanged(*panel, state*)

Callback passed to device list to redirect event to new event call

CallLightChanged(*panel, level*)

Callback passed to device list to redirect event to new event call

CallMotionDetected(*panel, state*)

Callback passed to device list to redirect event to new event call

CallSleepChanged(*panel, state*)

Callback passed to device list to redirect event to new event call

Click(*count=1, interval=None*)

Plays the default buzzer sound on all mirrored devices.

Parameters

- **count** (*int*) – number of buzzer sound to play. (Default value = 1)
- **interval** (*float*) – time gap between the starts of consecutive buzzer sounds. (Default value = None)

Notes

If count is greater than 1, interval must be provided.

property **DeviceAlias**

Defined Property to pass a list of Objects' DeviceAlias property

Returns list of the device aliases for all devices

Return type list of str

property **DisplayState**

Defined Property to pass Display State

Returns the current display state of the all devices ('On', 'Off')

Return type str

Notes

Display timer is applicable to TLI only.

property **DisplayTimer**

Defined Property to pass DisplayTimer

Returns Return list of display timer timeouts in seconds for all devices
Return type int

property **DisplayTimerEnabled**

Defined Property to pass DisplayTimerEnabled

Returns current state of the display timer for all devices
Return type bool

property **FirmwareVersion**

Defined Property to pass a list of Objects' FirmwareVersion property

Returns list of the firmware version for all devices
Return type list of str

GetHDCPStatus(*videoInput*)

Returns a list of the current HDCP Status for the given input on all devices.

Parameters **videoInput** (*str*) – input ('HDMI' or 'XTP').
Returns list of True or False
Return type list of bool

GetInputPresence(*videoInput*)

Returns a list of the current Input Presence for the given input on all devices.

Parameters **videoInput** (*str*) – input ('HDMI' or 'XTP').
Returns list of True or False
Return type list of bool

GetMute(*name*)

Returns the current mute state for the given channel name on all devices.

Parameters **name** (*str*) – name of channel ('Master', 'Speaker', 'Line', 'Click', 'Sound', 'HDMI' and 'XTP').
Returns 'On' or 'Off'
Return type str

GetVolume(*name*)

Returns the current volume level for the given channel name on all devices.

Parameters **name** (*str*) – name of channel ('Master', 'Click', 'Sound', 'HDMI' and 'XTP').
Returns Volume Level (0-100)
Return type int

property **HDCPStatusChanged**

Passback the HDCPStatusChanged Event

Event: Triggers when HDCP Status changes.

The callback takes two arguments. The first one is the MirrorUIDevice instance triggering the event and state with a tuple: (**Input**, **Status**).

Examples

```
@event(MirroredTLPs, 'HDCPStatusChanged')
def HandleHDCPStatusChanged(mirroredt1p, state):
    if state[0] == 'HDMI' and not state[1]:
        PodiumTLP.ShowPopup('No HDCP')
```

HideAllPopups()

Dismisses all popup pages on all devices.

HidePopup(*popup*)

Hide popup page on all devices.

Parameters **popup** (*int, str*) – popup page number or name.

HidePopupGroup(*group*)

Hide all popup pages in a popup group on all devices.

Parameters **group** (*int*) – popup group number.

property **Hostname**

Defined Property to pass list of Objects' Hostname property

Returns list of the hostname for all devices

Return type list of str

property **IPAddress**

Defined Property to pass a list of Objects' IPAddress property

Returns list of the IP address for all devices

Return type list of str

property **InactivityChanged**

Passback the InactivityChanged Event

Event: Triggers at times specified by SetInactivityTime() after state transition of inactivity timer.

The callback takes two arguments. The first one is the MirrorUIDevice instance triggering the event and time with a float value of inactivity time in seconds.

Examples

```
@event(MirroredTLPs, 'InactivityChanged')
def UnoccupyRoom(mirrotlp, time):
    if time == 3000:
        ShowWarning()
    else:
        ShutdownSystem()
```

property **InactivityTime**

Defined Property to pass a list of Objects' InactivityTime property

Returns list of seconds since last activity for the all devices.

Return type list of int

Notes

0 = Active, Nonzero = Time of inactivity.

property **InputPresenceChanged**

Passback the InputPresenceChanged Event

Event: Triggers when Input Presence changes.

The callback takes two arguments. The first one is the MirrorUIDevice instance triggering the event and state with a tuple: (**Input**, **Status**).

Examples

```
@event(MirroredTLPs, 'InputPresenceChanged')
def HandleInputPresenceChanged(mirrotlp, state):
    if state[0] == 'HDMI' and not state[1]:
        if mirrotlp.GetInputPresence('XTP'):
            mirrotlp.SetInput('XTP')
        else:
            mirrotlp.ShowPopup('No Input Available')
```

property **LidChanged**

Passback the LidChanged Event

Event: Triggers when the Lid state changes.

The callback takes two arguments. The first one is the MirrorUIDevice instance triggering the event and the second is the current lid state ('Opened' or 'Closed').

Examples

```
@event(MirroredTLPs, 'LidChanged')
def HandleHDCPStatusChanged(mirroredt1p, state):
    if state == 'Closed':
        ShutdownSystem()
```

property **LidState**

Defined Property to pass a list of Objects' LidState property

Returns list of the current lid state for all devices ('Opened' or 'Closed')

Return type list of str

property **LightChanged**

Passback the LightChanged Event

Event: Triggers when ambient light changes.

The callback takes two arguments. The first one is the MirrorUIDevice instance triggering the event and the second is the ambient light level in the range of 0 to 255.

Examples

```
@event(MirroredTLPs, 'LightChanged')
def HandleHDCPStatusChanged(mirroredt1p, level):
    print('{} Light Level Changed: {}'.format(mirroredt1p.DeviceAlias, level))
```

property **LightDetectedState**

Defined Property to pass a list of Objects' LightDetectedState property

Returns list of the state of light detection for all devices ('Detected' or 'Not Detected')

Return type list of str

property **MACAddress**

Defined Property to pass a list of Objects' MACAddress property

Returns list of the MAC address for all devices

Return type list of str

property **ModelName**

Defined Property to pass a list of Objects' ModelName property

Returns list of the model name for all devices

Return type list of str

property **MotionDecayTime**

Defined Property to pass MotionDecayTime

Returns period of time to trigger *MotionDetected* event after last motion was detected for all devices. The default (and minimum) value is 10 seconds.

Return type int

property **MotionDetected**

Passback the MotionDetected Event

Event: Triggers when Motion is detected.

The callback takes two arguments. The first one is the MirrorUIDevice instance triggering the event and the second one is a string ('Motion' or 'No Motion').

Examples

```
@event(MirroredTLPs, 'MotionDetected')
def HandleHDCPStatusChanged(mirroredt1p, state):
    if state == 'Motion':
        mirroredt1p.Wake()
```

property **MotionState**

Defined Property to pass a list of Objects' MotionState property

Returns list of the state of the Motion sensor for all devices (e.g. 'Motion', 'No Motion')

Return type list of str

property **Objects**

Defined Property to pass a list of Objects in the mirrored group

Returns list of UIDevice objects defined in the class

Return type list of extronlib.device.UIDevice

property **PartNumber**

Defined Property to pass a list of Objects' PartNumber property

Returns list of the part numbers for all devices

Return type list of str

PlaySound(filename)

Play a sound file identified by filename on all devices.

Parameters **filename** (str) – name of sound file.

Notes

- Only WAV files can be played.
- A subsequent call will preempt the currently playing file.
- Sound file must be added to the project file.

property **SerialNumber**

Defined Property to pass a list of Objects' SerialNumber property :returns: list of the serial numbers for all devices :rtype: list of str

SetAutoBrightness(state)

Set the auto brightness state on all devices. Either 'On' or True turns on auto brightness. 'Off' or False turns off auto brightness.

Parameters **state** (bool, str) – whether to enable auto brightness

SetBrightness(level)

Set the LCD screen brightness level on all devices.

Parameters **level** (int) – brightness level from 0 to 100

SetDisplayTimer(*state, timeout*)

Enables or Disables the display timer on all devices. Either 'On' or True enables display timer. 'Off' or False disables display timer.

Parameters

- **state** (*bool, str*) – whether to enable the display timer
- **timeout** (*int*) – time in seconds before turn off the display

Notes

Display timer is applicable to TLI only.

SetInactivityTime(*times*)

Set the inactivity times when the InactivityChanged event will trigger for all devices.

Parameters

- **times** (*list of ints*) – list of times. Each time in whole seconds

SetInput(*videoInput*)

Set the video input on all devices.

Parameters

- **videoInput** (*str*) – input to select ('HDMI' or 'XTP')

SetLEDBlinking(*ledID, rate, stateList*)

Set the LED cycle at ADA compliant rates through a list of states on all devices.

Parameters

- **ledID** (*int*) – LED id (65533)
- **rate** (*str*) – ADA compliant blink rate. ('Slow', 'Medium', 'Fast')
- **stateList** (*list of str*) – List of colors ('Off', 'Green', 'Red')

SetLEDState(*ledID, state*)

Make the LED to a given color on all devices.

Parameters

- **ledID** (*int*) – LED id (65533)
- **state** (*str*) – LED Color ('Off', 'Green', 'Red')

SetMotionDecayTime(*duration*)

Set the period of time for the MotionDetected event to trigger for all devices.

Parameters

- **duration** (*float*) – time in seconds (minimum/default value is 10)

SetMute(*name, mute*)

Sets the mute state for the given channel on all devices.

Parameters

- **name** (*str*) – name of channel ('Master', 'Speaker', 'Line', 'Click', 'Sound', 'HDMI' and 'XTP').
- **mute** (*str*) – mute state ('On', 'Off')

SetSleepTimer(*state, duration=None*)

Enables or Disables the sleep timer on all devices. Either 'On' or True enables sleep timer. 'Off' or False disables sleep timer.

Parameters

- **state** (*bool, str*) – whether to enable the display timer
- **duration** (*int*) – time in seconds to sleep (Default value = None)

SetVolume(*name, level*)

Sets the volume level for the given channel name on all devices.

Parameters

- **name** (*str*) – name of channel ('Master', 'Click', 'Sound', 'HDMI' and 'XTP').
- **level** (*int*) – volume level 0 to 100

SetWakeOnMotion(*state*)

Enables or disables wake on motion on all devices. Either 'On' or True turns on wake on motion. 'Off' or False turns off wake on motion.

Parameters

- **state** (*bool, str*) – whether to enable wake on motion

ShowPage(*page*)

Show page on the screen on all devices.

Parameters **page** (*int, str*) – absolute page number or name

ShowPopup(*popup, duration=0*)

Display pop-up page on all devices for a period of time

Parameters • **popup** (*int, str*) – absolute pop-up page number or name
 • **duration** (*float*) – duration (in seconds) the pop-up remains on the screen. 0 means forever. (Default value = 0)

Notes

If a pop-up is already showing for a finite period of time, calling this method again with the same pop-up will replace the remaining period with the new period.

Sleep()

Forces all devices to sleep immediately

property SleepChanged

Passback the SleepChanged Event

Event: Triggers when sleep state changes.

The callback takes two arguments. The first one is the MirrorUIDevice instance triggering the event and the second one is a string ('Asleep' or 'Awake').

Examples

```
@event(MirroredTLPs, 'SleepChanged')
def HandleSleepChanged(mirror_tlp, state):
    if state[0] == 'Awake' and not state[1]:
        print('{} Sleep State Changed: {}'.format(mirror_tlp.DeviceAlias, state))
```

property SleepState

Defined Property to pass a list of Objects' SleepState property

Returns list of the current sleep state for all devices (e.g. 'Asleep', 'Awake')

Return type list of str

property SleepTimer

Defined Property to pass SleepTimer

Returns sleep timer timeout for all devices.

Return type int

property SleepTimerEnabled

Defined Property to pass SleepTimerEnabled

Returns current state of the sleep timer for all devices. **True** if enabled. **False** if disabled.

Return type bool

StopSound()

Stop playing sound file on all devices

property Usage

Defined Property to pass a list of Objects' Usage property

Returns list of user data usage for all devices in KB (**used, total**).

Return type list of tuples of ints

Wake()

Forces all devices to wake up immediately

property **WakeOnMotion**

Defined Property to pass WakeOnMotion

Returns current wake on motion state for all devices. **True** if enabled. **False** if disabled.

Return type bool

MirroreBUSDevice Class

class **MirroreBUSDevice**(*eBUSList*)

This class creates an object that contains a list of eBUSDevice objects to be used with the Button, Label, Level and Knob classes defined in this file. The class object is also used in place of the eBUSDevice object for eBUS panel operations such as timer settings and mutes.

Initializes the module's MirroreBUSDevice in program.

Parameters **eBUSList** (*list of extronlib.device.eBUSDevice*) – List of eBUSDevice Objects

Examples

```
# Comment out the extronlib.ui import statement
#from extronlib.ui import Button, Knob, Level

# Add the import statement for the MirrorUI file in System Manager
from MirrorUI import MirroreBUSDevice, Button, Knob, Level

EBP1 = eBUSDevice(MainProcessor, 'eBUSAlias')
EBP2 = eBUSDevice(MainProcessor, 'eBUSAlias_2')

# Add the statement defining the MirroreBUSDevice and program using normal
# methods and events
MirroredEBPs = MirroreBUSDevice([EBP1, EBP2])
```

CallInactivityChanged(*panel, time*)

Callback passed to device list to redirect event to new event call

CallLidChanged(*panel, state*)

Callback passed to device list to redirect event to new event call

CallSleepChanged(*panel, state*)

Callback passed to device list to redirect event to new event call

Click(*count=1, interval=None*)

Play default buzzer sound on all devices

Parameters • **count** (*int*) – number of buzzer sound to play. (Default value = 1)
 • **interval** (*float*) – time gap between the starts of consecutive buzzer sounds. (Default value = None)

Notes

If count is greater than 1, interval must be provided.

property **DeviceAlias**

Defined Property to pass a list of Objects' DeviceAlias property

Returns list of the device aliases for all devices
Return type list of str

GetMute(*name*)

Returns the current mute state for the given channel name on all devices.

Parameters **name** (*str*) – name of channel ('Click').
Returns 'On' or 'Off'
Return type str

property **Host**

Defined Property to pass a list of Objects' Host property

Returns list of the Extron ProcessorDevice to which the eBUSDevice is connected for all devices.
Return type list of extronlib.device.ProcessorDevice

property **ID**

Defined Property to pass ID

Returns the device's ID (set by DIP switch) for all devices
Return type int

property **InactivityChanged**

Passback the InactivityChanged Event

Event: Triggers at times specified by SetInactivityTime() after state transition of inactivity timer.

The callback takes two arguments. The first one is the MirroreBUSDevice instance triggering the event and time with a float value of inactivity time in seconds.

Examples

```
@event(MirroredEBPs, 'InactivityChanged')
def HandleInactivityChanged(mirrorebp, time):
    if time == 3000:
        ShowWarning()
    else:
        ShutdownSystem()
```

property **InactivityTime**

Defined Property to pass a list of Objects' InactivityTime property

Returns list of seconds since last activity for the all devices.
Return type list of int

Notes

0 = Active, Nonzero = Time of inactivity.

property **LidChanged**

Passback the LidChanged Event

Event: Triggers when the Lid state changes.

The callback takes two arguments. The first one is the MirroreBUSDevice instance triggering the event and the second is the current lid state ('Opened' or 'Closed').

Examples

```
@event(MirroredEBPs, 'LidChanged')
def HandleLidChanged(mirrorebp, state):
    if state == 'Closed':
        ShutdownSystem()
```

property **LidState**

Defined Property to pass value of Objects[0]'s LidState property

Returns list of the current lid state for all devices ('Opened' or 'Closed')
Return type list of str

property **ModelName**

Defined Property to pass a list of Objects' ModelName property :returns: list of the model name for all devices :rtype: list of str

property **Objects**

Defined Property to pass a list of Objects in the mirrored group :returns: list of eBUSDevice objects defined in the class :rtype: list of extronlib.device.eBUSDevice

property **PartNumber**

Defined Property to pass a list of Objects' PartNumber property :returns: list of the part numbers for all devices :rtype: list of str

SetInactivityTime(*times*)

Set the inactivity times when the InactivityChanged event will trigger for all devices.

Parameters **times** (*list of ints*) – list of times. Each time in whole seconds

SetMute(*name, mute*)

Set the mute state for the given channel name on all devices.

Parameters • **name** (*str*) – name of channel ('Click').
 • **mute** (*str*) – mute state ('On', 'Off')

SetSleepTimer(*state, duration=None*)

Enables or Disables the sleep timer on all devices. Either 'On' or **True** enables sleep timer. 'Off' or **False** disables sleep timer.

Parameters • **state** (*bool, str*) – whether to enable the display timer
 • **duration** (*int*) – time in seconds to sleep (Default value = None)

Sleep()

Forces all devices to sleep immediately

property **SleepChanged**

Passback the SleepChanged Event

Event: Triggers when sleep state changes.

The callback takes two arguments. The first one is the MirroredDevice instance triggering the event and the second one is a string ('Asleep' or 'Awake').

Examples

```
@event(MirroredEBPs, 'SleepChanged')
def HandleSleepChanged(mirroredbp, state):
    if state[0] == 'HDMI' and not state[1]:
        print('{} Sleep State Changed: {}'.format(mirroredbp.DeviceAlias, state))
```

property **SleepState**

Defined Property to pass a list of Objects' SleepState property

Returns list of the current sleep state for all devices (e.g. 'Asleep', 'Awake')
Return type list of str

property **SleepTimer**

Defined Property to pass SleepTimer :returns: sleep timer timeout for all devices. :rtype: int

property **SleepTimerEnabled**

Defined Property to pass a list of Objects' SleepTimerEnabled property

Returns current state of the sleep timer for all devices. **True** if enabled. **False** if disabled.
Return type bool

Wake()

Forces all devices to wake up immediately

Button Class

class **Button**(*UIHost, ID, holdTime=None, repeatTime=None*)

This class takes either a singular UIDevice or eBUSDevice or a MirrorUIDevice or MirroredBUSDevice object and handles the properties, methods and events for all Button Objects created within this class.

Representation of Hard/Soft buttons.

Parameters

- **UIHost** (*see note.*) – device or mirrored devices hosting this UI Object.
- **ID** (*int, str*) – ID or Name of the UIObject.
- **holdTime** (*float*) – Time for Held event.
- **repeatTime** (*float*) – Time for Repeated event.

Notes

- This class is a direct replacement for the extronlib.ui.Button class.
- All properties, methods and events in extronlib.ui.Button are in this class and operate in the same fashion.
- The UIHost parameter can take objects of type:
 - extronlib.device.eBUSDevice
 - extronlib.device.UIDevice
 - MirrorUI.MirroredBUSDevice
 - MirrorUI.MirrorUIDevice

Examples

```
# Create an instance of button ID 101 for all panels
powerOn = Button(MirroredTLPs, 101)
```

property **BlinkState**

Defined Property to pass BlinkState

Returns current blink state ('Not blinking' or 'Blinking') for all devices.
Return type str

CallHeld(*button, state*)

Callback passed to device list to redirect event to new event call

CallPressed(*button, state*)

Callback passed to device list to redirect event to new event call

CallReleased(*button, state*)

Callback passed to device list to redirect event to new event call

CallRepeated(*button, state*)

Callback passed to device list to redirect event to new event call

CallTapped(*button, state*)

Callback passed to device list to redirect event to new event call

CustomBlink(*rate, stateList*)

Makes the buttons on all devices cycle through each of the states provided.

Parameters

- **rate** (*float*) – duration of time in seconds for one visual state to stay until replaced by the next visual state.
- **stateList** (*list of ints*) – list of visual states that this button blinks among.

property **Enabled**

Defined Property to pass Enabled

Returns 'True' if the control object is enabled else 'False' for all devices.
Return type bool

property **Held**

Passback the Held Event

Event: Get/Set the callback when hold expire event is triggered

The callback function must accept exactly two parameters, which are the mirrored Button that triggers the event and the state (e.g. 'Held').

Examples

```
@event(showSetupPage, 'Held')
def ShowSetupPage(mirrorbutton, state):
    MirroredTLPs.ShowPage('Setup Page')
```

property **Host**

Defined Property to pass a list of Objects' Host property

Returns list of the UIDevice objects that host this control object.

Return type list of extronlib.ui.Button

Notes

This is a deviation from the extronlib.ui.Button class. In this class a list of the Hosts for each Button object created is returned in a list of UIDevice or eBUSDevice objects

property **ID**

Defined Property to pass ID

Returns the ID for the object on all devices

Return type int

property **Name**

Defined Property to pass Name

Returns the object's Name on all devices

Return type str

property **Objects**

Defined Property to pass a list of Objects in the mirrored group

Returns list of Button objects defined in the class

Return type list of extronlib.ui.Button

property **Pressed**

Passback the Pressed Event

Event: Get/Set the callback when the button is pressed

The callback function must accept exactly two parameters, which are the mirrored Button that triggers the event and the state (e.g. 'Pressed').

Examples

```
@event(powerOn, 'Pressed')
def PowerOn(mirrorbutton, state):
    mirrorbutton.SetBlinking('Medium', [2, 4])
    mainProjector.Send('POWER ON')
```

property **PressedState**

Defined Property to pass a list of Objects' PressedState property

Returns the mirror device objects' pressed state. **True** if and button in the group is pressed, else **False**

Return type bool

property **Released**

Passback the Released Event

Event: Get/Set the callback when the button is released

The callback function must accept exactly two parameters, which are the mirrored Button that triggers the event and the state (e.g. 'Released').

Examples

```
@event(closeSetupPage, 'Released')
def ClosePage(mirrorbutton, state):
    MirroredTLPs.ShowPage('Main')
```

property **Repeated**

Passback the Repeated Event

Event: Get/Set the callback when repeat event is triggered

The callback function must accept exactly two parameters, which are the mirrored Button that triggers the event and the state (e.g. 'Repeated').

Examples

```
@event(volumeUp, 'Repeated')
def VolUp(mirrorbutton, state):
    global VolLevel
    if VolLevel < 20:
        VolLevel += 1
        mainProjector.Send('VOLUME {}'.format(VolLevel))
```

SetBlinking(rate, stateList)

Makes the buttons on all devices cycle, at ADA compliant rates, through each of the states provided.

Parameters

- **rate** (*str*) – ADA compliant blink rate. ('Slow', 'Medium', 'Fast')
- **stateList** (*list of ints*) – list of visual states that this button blinks among.

SetEnable(enable)

Enable or disable a UI control object on all devices

Parameters

- **enable** (*bool*) – True to enable the object or **False** to disable it.

SetState(state)

Set the current visual state on all devices

Parameters

- **state** (*int*) – visual state number.

SetText(text)

Specify text to display on the UIObject on all devices

Parameters

- **text** (*str*) – text to display.

SetVisible(visible)

Change the visibility of a UI control object on all devices

Parameters

- **visible** (*bool*) – True to make the object visible or **False** to hide it.

property **State**

Defined Property to pass State

Returns

the object's visual state number on all devices

Return type

int

property **Tapped**

Passback the Tapped Event

Event: Get/Set the callback when tap event is triggered

The callback function must accept exactly two parameters, which are the mirrored Button that triggers the event and the state (e.g. 'Tapped').

Examples

```
@event(zoomIn, 'Tapped')
def ZoomInTapped(mirrorbutton, state):
    PTZ.Send('zoom +1')    # Nudge the zoom
```

property **Visible**

Defined Property to pass a list of Objects' Visible property

Returns the object's visible state for all devices. **True** if visible, else **False**

Return type bool

Knob Class

class **Knob**(*UIHost, ID, holdTime=None, repeatTime=None*)

This class takes either a singular UIDevice or eBUSDevice or a MirrorUIDevice or MirroredBUSDevice object and handles the properties, methods and events for all Knob Objects created within this class.

Knob is a rotary control that has 36 steps for a full revolution.

Parameters

- **UIHost** (*see note.*) – device or mirrored devices hosting this UI Object.
- **ID** (*int, str*) – ID or Name of the UIObject.

Notes

- This class is a direct replacement for the extronlib.ui.Knob class.
- All properties, methods and events in extronlib.ui.Knob are in this class and operate in the same fashion.
- The UIHost parameter can take objects of type:
 - extronlib.device.eBUSDevice
 - extronlib.device.UIDevice
 - MirrorUI.MirroredBUSDevice
 - MirrorUI.MirrorUIDevice

Examples

```
# Creates an instance of the knob for all eBUS panels  
EBPKnob = Knob(MirroredEBPs, 61001)
```

*Call***Turned**(*knob, direction*)

Callback passed to device list to redirect event to new event call

property **Host**

Defined Property to pass a list of Objects' Host property

Returns list of the UIDevice objects that host this control object.
Return type list of extronlib.ui.Knob

Notes

This is a deviation from the extronlib.ui.Knob class. In this class a list of the Hosts for each Knob object created is returned in a list of UIDevice or eBUSDevice objects

property **ID**

Defined Property to pass ID

Returns the ID for the object on all devices
Return type int

property **Objects**

Defined Property to pass a list of Objects in the mirrored group

Returns list of Knob objects defined in the class
Return type list of extronlib.ui.Knob

property **Turned**

Passback the Turned Event

Event: Get/Set callback when knob is turned

The callback takes two parameters. The first one is the mirrored Knob itself and the second one is a signed integer indicating steps that was turned. Positive values indicate clockwise rotation.

Examples

```
@event(mainKnob, 'Turned')
def handleMainKnob(mirrorknob, direction):
    if direction > 0:
        for i in range(direction):
            mainProjector.Send('VOLUME UP')
    else:
        for i in range(direction, 0):
            mainProjector.Send('VOLUME DOWN')
```

Label Class

class **Label1**(*UIHost, ID*)

This class takes either a singular UIDevice or eBUSDevice or a MirrorUIDevice or MirroredBUSDevice object and handles the properties and methods for all Label Objects created within this class.

Label object displays text string on the screen.

Parameters

- **UIHost** (*see note.*) – device or mirrored devices hosting this UI Object.
- **ID** (*int, str*) – ID or Name of the UIObject.

Notes

- This class is a direct replacement for the extronlib.ui.Label class.
- All properties, methods and events in extronlib.ui.Label are in this class and operate in the same fashion.
- The UIHost parameter can take objects of type:
 - extronlib.device.UIDevice
 - MirrorUI.MirrorUIDevice

Examples

```
# Creates an instance of label ID 3001 for all panels
RoomLabel = Label(MirroredTLPs, 3001)
```

property **Host**

Defined Property to pass a list of Objects' Host property

Returns list of the UIDevice and eBUSDevice objects that host this control object.
Return type list of extronlib.device.UIDevice, extronlib.device.eBUSDevice

Notes

This is a deviation from the extronlib.ui.Label class. In this class a list of the Hosts for each Label object created is returned in a list of UIDevice and/or eBUSDevice objects.

property **ID**

Defined Property to pass ID

Returns list of the ID for the object on all devices
Return type list of int

property **Name**

Defined Property to pass Name

Returns the object's Name on all devices
Return type str

property **Objects**

Defined Property to pass a list of Objects in the mirrored group

Returns list of Label objects defined in the class
Return type list of extronlib.ui.Label

SetText(*text*)

Specify text to display on the UIObject for all devices.

Parameters **text** (*str*) – text to display.

SetVisible(*visible*)

Change the visibility of a UI control object on all devices

Parameters **visible** (*bool*) – True to make the object visible or **False** to hide it.

property **Visible**

Defined Property to pass Visible

Returns the object's visible state for all devices. **True** if visible, else **False**

Return type bool

Level Class

class **Level**(*UIHost, ID*)

This class takes either a singular UIDevice or eBUSDevice or a MirrorUIDevice or MirroredBUSDevice object and handles the properties and methods for all Level Objects created within this class.

This module defines interfaces of Level UI.

Parameters

- **UIHost** (*see note.*) – device or mirrored devices hosting this UI Object.
- **ID** (*int, str*) – ID or Name of the UIObject.

Notes

- This class is a direct replacement for the extronlib.ui.Level class.
- All properties, methods and events in extronlib.ui.Level are in this class and operate in the same fashion.
- The UIHost parameter can take objects of type:
 - extronlib.device.eBUSDevice
 - extronlib.device.UIDevice
 - MirrorUI.MirroredBUSDevice
 - MirrorUI.MirrorUIDevice

Examples

```
# Creates an instance of level ID 4001 for all panels  
VolumeLevel = Level(MirroredTLPs, 4001)
```

Dec()

Nudge the level down a step for all devices

property Host

Defined Property to pass a list of Objects' Host property

Returns list of the UIDevice and eBUSDevice objects that host this control object.
Return type list of extronlib.device.UIDevice, extronlib.device.eBUSDevice

Notes

This is a deviation from the extronlib.ui.Level class. In this class a list of the Hosts for each Level object created is returned in a list of UIDevice and eBUSDevice objects.

property ID

Defined Property to pass ID

Returns the ID for the object on all devices
Return type int

Inc()

Nudge the level up a step for all devices

property Level

Defined Property to pass Level

Returns the object's current level for all devices
Return type int

property Max

Defined Property to pass Max

Returns the object's upper bound of the level for all devices
Return type int

property **Min**

Defined Property to pass Min

Returns the object's lower bound of the level for all devices
Return type int

property **Name**

Defined Property to pass Name

Returns the object's Name on all devices
Return type str

property **Objects**

Defined Property to pass a list of Objects in the mirrored group

Returns list of Label objects defined in the class
Return type list of extronlib.ui.Level

SetLevel(*level*)

Set the current level for all devices

Parameters **level** (*int*) – Discrete value of the level object.

SetRange(*Min, Max, Step=1*)

Set level object's allowed range and the step size for all devices

Parameters

- **Min** (*int*) – Minimum level
- **Max** (*int*) – Maximum level
- **Step** (*int*) – Optional step size for Inc() and Dec(). (Default value = 1)

Notes

- The default range is 0 - 100 with a step size of 1.
- For multi-state levels, you must set the level range to match the number of states in the level.

SetVisible(*visible*)

Change the visibility of a UI control object for all devices

Parameters **visible** (*bool*) – True to make the object visible or **False** to hide it.

property **Visible**

Defined Property to pass Visible

Returns the object's visible state for all devices. **True** if visible, else **False**
Return type bool

Slider Class

class **Slider**(UIHost, ID)

This class takes either a singular UIDevice or a MirrorUIDevice object and handles the properties and methods for all Level Objects created within this class.

This module defines interfaces of Slider UI.

Parameters

- **UIHost** (*see note.*) – device or mirrored devices hosting this UI Object.
- **ID** (*int, str*) – ID or Name of the UIObject.

Notes

- This class is a direct replacement for the extronlib.ui.Slider class.
- All properties, methods and events in extronlib.ui.Slider are in this class and operate in the same fashion.
- The UIHost parameter can take objects of type:
 - extronlib.device.UIDevice
 - MirrorUI.MirrorUIDevice

Examples

```
# Creates an instance of Slider ID 4001 for all panels
VolumeSlider = Slider(MirroredTLPs, 4001)
```

CallChanged(slider, state, value)

Callback passed to device list to redirect event to new event call

CallPressed(slider, state, value)

Callback passed to device list to redirect event to new event call

CallReleased(slider, state, value)

Callback passed to device list to redirect event to new event call

property **Changed**

Passback the Changed Event

Event: Get/Set the callback when the slider thumb position is changed.

The callback takes three arguments. The first is the mirrored Slider instance triggering the event, the second is the state, and the third is the new slider value.

Examples

```
@event(ProgramAudio, 'Changed')
def handleProgramAudio(mirrorslider, state, value):
    if state == 'Changed':
        # Send new program audio level
```

property **Enabled**

Defined Property to pass Enabled

Returns the object's current Enabled for all devices
Return type int

property **Fill**

Defined Property to pass Fill

Returns the object's current Fill for all devices
Return type int

property **Host**

Defined Property to pass a list of Objects' Host property

Returns list of the UIDevice and eBUSDevice objects that host this control object.
Return type list of extronlib.device.UIDevice, extronlib.device.eBUSDevice

Notes

This is a deviation from the extronlib.ui.Slider class. In this class a list of the Hosts for each Slider object created is returned in a list of UIDevice and eBUSDevice objects.

property **ID**

Defined Property to pass ID

Returns the ID for the object on all devices
Return type int

property **Max**

Defined Property to pass Max

Returns the object's upper bound of the fill for all devices
Return type int

property **Min**

Defined Property to pass Min

Returns the object's lower bound of the fill for all devices
Return type int

property **Name**

Defined Property to pass Name

Returns the object's Name on all devices
Return type str

property **Objects**

Defined Property to pass a list of Objects in the mirrored group

Returns list of Slider objects defined in the class
Return type list of extronlib.ui.Slider

property **Pressed**

Passback the Pressed Event

Event: Get/Set the callback when the slider is pressed.

The callback takes three arguments. The first is the mirrored Slider instance triggering the event, the second is the state, and the third is the new slider value.

Examples

```
@event(ProgramAudio, 'Pressed')
def handleProgramAudio(mirrorslider, state, value):
    if state == 'Pressed':
        # Send new program audio level
```

property **Released**

Passback the Released Event

Event: Get/Set the callback when the slider is released.

The callback takes three arguments. The first is the mirrored Slider instance triggering the event, the second is the state, and the third is the new slider value.

Examples

```
@event(ProgramAudio, 'Released')
def handleProgramAudio(mirrorslider, state, value):
    if state == 'Released':
        # Send new program audio level
```

SetEnable(*enable*)

Enable or disable a UI control object on all devices

Parameters **enable** (*bool*) – True`` to enable the object or **False** to disable it.

SetFill(*Fill*)

Set the current fill level for all devices

Parameters **Fill** (*int*) – Discrete value of the slider object.

SetRange(*Min, Max, Step=1*)

Set slider object's allowed range and the step size for all devices

Parameters • **Min** (*int*) – Minimum fill
 • **Max** (*int*) – Maximum fill
 • **Step** (*int*) – Optional step size for Inc() and Dec(). (Default value = 1)

Notes

The default range is 0 - 100 with a step size of 1.

SetVisible(*visible*)

Change the visibility of a UI control object for all devices

Parameters **visible** (*bool*) – True`` to make the object visible or **False** to hide it.

property **Step**

Defined Property to pass Step

Returns the object's Step on all devices
Return type int

property **Visible**

Defined Property to pass Visible

Returns the object's visible state for all devices. **True** if visible, else **False**
Return type bool